

J Primer

作者: Eric Iverson 翻译: Taigua

目录

1 从这里开始	3
2 为什么学J?	4
3 本书的目标	6
4 需要的背景知识	7
5 如何使用本书	8
6 开始	9
7 实验	10
8 标准配置	12
9 术语	13
10 字母表	14
11 单词	16
12 句子	17
13 动词	18
14 名词	19
15 数字	20

16 负数	22
17 原语	24

第 1 章

从这里开始

J是一种通用的高级编程语言。如果您是J的新手，并且想成为一名J程序员，那么这是一个很好的起点。即使您有相当丰富的编程经验，J也有很多独特之处，在深入研究之前，至少浏览一下这些内容是值得的。

第 2 章

为什么学J?

J是一种非常丰富的语言。你可能学习和使用它很多年，但仍然认为自己是初学者。这与Basic或Java等更简单的语言形成鲜明对比，在这些语言中，数月的努力学习和使用将使您成为专家。成为一名专家J程序员所需要的努力与成为一名专家c++程序员所需要的努力更接近。

好消息是，J的本质是如此简单和一致，你可以很快地学会解决真正有趣的问题。

学习足够的Basic或Java来解决琐碎的问题更容易，但学习足够的J来解决更有趣和更具挑战性的问题更容易。一旦你掌握了这种水平的技能，你就不会走到路的尽头，而是可以继续提高，使自己成为一个更好、更强大的程序员。

J尤其擅长对阵列数据进行数学、统计和逻辑方面的分析。它是一个强大的工具，可以为老问题构建新的更好的解决方案，甚至可以更好地找到尚未完全理解的问题的解决方案。

作为一种通用编程语言，J系统还提供：

- 两个集成的开发环境。JHS和JQt
- 标准库、实用程序和包
- 为应用程序构建事件驱动图形用户界面（JQt）
- 几种与其他编程语言和应用程序接口的方法
- 快速应用程序原型和开发
- 免版税发布应用程序的运行时版本

如果你对具有挑战性的数据处理问题的编程解决方案感兴趣，那么在学习J
上投入的时间将是值得的。

第 3 章

本书的目标

[NuVoc](#)是J语言的权威和明确的规范。[J字典](#)也可以用来学习J，但正因为它简洁、完整且严谨地涵盖了这门语言，并且更侧重于复杂之处而非寻常之处，它确实把我们中的一些人吓跑了。

本书为初学者提供了一个更亲切、更温和的起点。它带着你沿着一条路径，以轻松的步骤逐步前进，直到你能够用J语言编写应用程序。在这个过程中，你会通过简化和具体的情境来接触J的所有关键概念。这本手册的目的是让你快速上手，达到能够使用NuVoc和J字典的程度，并且用起来会让你纳闷：当初为什么还要为这些简单的东西费心呢？

你应该能够相当快地完成入门，最后将成为入门级的J程序员。这样，你将比最有经验的Basic或Java程序员拥有更强大的编程能力。

第 4 章

需要的背景知识

本书假设你熟悉另一种编程语言，如Basic、Java或C。然而，这不是先决条件，即使J是你的第一门计算机语言，应该也不会有什么特别的问题（实际上，更应祝贺你！）。

大多数事情都可以在J中完成，就像在其他语言中完成一样，并且通常一个主题的引入方式和在其他语言中引入它一样，这使得在J中更容易理解它是如何工作的。在某些情况下，有一种更好的J方法来解决这个问题，也会介绍这种方法。

第 5 章

如何使用本书

本书是一系列小的，易消化的部分，适合从头到尾按顺序阅读。一个章节通常需要你读过大部分甚至所有之前的章节。跳来跳去毫无意义，而且可能令人沮丧。

本书是自包含的，可以在不访问J系统的情况下读取。特别是，与J系统交互的示例显示了输入的内容以及系统如何响应。然而，它的目的是在访问系统和尽可能多地使用系统的情况下读取。强烈建议你最后输入所有示例，并尽可能多地把玩它们的变体。

有时章节使用的术语和概念直到后面才定义。这就要求你在第一次阅读时有一个初步的理解并继续，在第二次阅读时它们将变得更加具体。

本书最好读三遍：

- 浏览整本书。可以尝试一些例子，但最好持续前进，了解全局。
- 第二次仔细阅读并尝试所有的例子。
- 第三次尝试自己的例子来澄清你的理解，并提升你实际使用系统机制的舒适度（而不仅仅是阅读它和遵循说明）。

第 6 章

开始

本章假设你在 [JQt IDE](#)（交互式开发环境）中使用 [最新版本的J](#)。

Term窗口是一个执行窗口。你在Term窗口中键入J语句，当按Enter键时，J将执行它们并显示结果。

在Term窗口中键入以下行并按Enter键。

```
2 + 3
```

语句被执行并显示结果。输入以下行并按Enter键。

```
5 - 3
```

你的会话应该看起来像这样：

```
2 + 3
```

```
5
```

```
5 - 3
```

```
2
```

你输入的行缩进三个空格，J的答案从行首开始。

第 7 章

实验

我们鼓励你去尝试。尝试用不同的数字输入类似的行。很明显，+是加法，-是减法。输入一些用*表示乘，用%表示除的行。通过使用%，你将很快看到结果可以是2.5这样的数字，并且它们也可以用作参数。

在你有更多的经验之前，你有时可能会对你所观察到的东西感到惊讶甚至不安。一步一步来。尝试一些你确信你已经知道答案的例子，并做实验来证实你的理解。如果一个结果让你太困惑，早期阶段不要花时间在它上面。

3 - 5

_2

_2，而不是-2，可能会让你困惑。别担心，一会儿就会解释的。

本书中的大多数示例会显示你应输入的内容（缩进三个空格），同时也会显示系统返回的结果。这意味着你可以轻松地通读本入门手册，无需实际操作系统即可查看结果。然而，真正学会的唯一途径，最终还是靠你亲自尝试这些示例，并用自己的想法进行实验。示例采用等宽字体显示，与它们在系统Term窗口中的呈现方式基本一致。部分示例会使用较大的字体，以便你更轻松地阅读并在系统中输入——这通常出现在你可能因对某些词汇不熟悉而输错内容，或者输入错误可能导致令人困惑的结果时。

在做实验的时候，你经常想对你已经输入过的句子做一些细微的改变。有几个捷径可以使这更容易。在Term窗口中，你可以将光标移动到窗口中

的任意行，然后按Enter键，将该行作为窗口底部的新行，以便进行编辑。通过按住Ctrl+Shift并按向上箭头键，可以调出以前的输入行进行编辑，持续按向上箭头直到看到想要处理的行。

大多数小节中的示例都是独立的，但小节的后面部分可能依赖于前面部分中采取的步骤。有些小节依赖于前面几节中采取的步骤，但这应该是相当明显的。

第 8 章

标准配置

本书中的示例假设系统在JQt IDE中启动时运行标准配置文件。这将配置你的系统并提供一些标准实用程序。

在Term窗口中输入CR（大写字母C后跟大写字母R），以快速检查配置文件是否已运行。

```
CR
```

如果运行结果是空行，那么标准配置文件已经运行，你可以跳过本节的其余部分。

相反，如果你看到：

```
CR
```

```
|value error
```

那么你必须更改系统，以便运行标准配置文件。一个正常的J安装总是被配置为运行标准配置。 参考[安装J](#)。

第 9 章

术语

所有的编程语言都与英语有共同之处。在类比含义接近的情况下，J编程语言倾向于使用英语术语，而不是数学和其他编程语言中使用的术语。

像在其他语言中一样，你可以说“代码行”，但在J语言中，倾向于说“句子”。类似地，你可以把+称为“函数”，但在J中通常称为“动词”。

J使用的英语术语有：字母表、单词、句子、短语、动词、名词、副词和连词。

采用这种方法有几个原因。它解决的一个问题是：传统术语在数学和许多编程语言中有大量相关但又微妙不同的用法。例如：函数、子函数、运算符、程序、例程和子例程在不同的编程语言中都以略有不同的方式使用。J没有继承这种混乱，而是采用自己的术语，并在其上下文中精确地定义它们。

使用英语术语可以让你很好地了解术语在J中的一般含义。此外，使用自然语言术语可以鼓励和促进对问题进行英语陈述，并更直接地编写相应的J句子。

我们鼓励使用J术语，但肯定不是强制性的，使用术语“函数”而不是“动词”是完全可以接受的。

第 10 章

字母表

J编程语言的字母表是ASCII字母表，由以下部分组成：

26 lowercase letters (a to z)

26 uppercase letters (A to Z)

0 1 2 3 4 5 6 7 8 9

= < > _

+ * - %

^ \$ ~ |

. : , ;

! / \

[] { }

" ` @ & ?

()

,

有几个字符有时会引起混淆。-（减号）字符不同于_（下划线）字符，并且有三种不同的引号字符：

' quote

" double-quote

` back-quote

如果你尝试一个例子，输入"（双引号）误输入为''（两个单引号），你会感到失望，因为你的结果与书中的不一样。

句号.通常称为点。

第 11 章

单词

单词是字母表中有意义的一组字符。

2.5 + 5

这个句子有三个单词：数字2.5，+和数字5。

用字符组成句子中的单词的规则很简单，目前，常识足以识别句子中的单词。有一些复杂情况在后面的章节中讨论。

第 12 章

句子

句子是由一组单词组成的完整指令。

下面的2加5指令就是一个J语言的句子。

`2 + 5`

与英语句子不同，J语言的句子不以句号或其他标点符号结尾。

相反，J编程语言的句子通常是一个完整的行。

第 13 章

动词

在下面的句子中+是动词（表示动作的单词）。

2 + 5

第 14 章

名词

在下面的句子中，数字2和5都是名词。

$$2 + 5$$

第 15 章

数字

以下这些是数字：0 12 02.5 12.75 0.5 7e6

输入这些数字或其他数字，用动词+ - * %组成简单句子。

2 * 12.75

25.5

一个重要的规则是数字不能以点（.）开头。

0.5

0.5

0.5 + 3

3.5

.5

+-+--+

|.|5|

+-+--+

`.5 + 3`

| `syntax error`

| `.5+3`

显然，`.5`和`0.5`不一样。你现在不需要知道`.5`是什么，但重要的是明白数字不是以点开头的。

`_`（下划线）是无穷大，是一个数字。

第 16 章

负数

负数用_（下划线）表示，而不是-（减号）。

5 + _3

2

_7

_7

_5e_3

_0.005

5 - 9

_4

去掉-和9之间的空白并不会改变含义。

5 -9

_4

-总是动词。这简化了识别句子的规则，因为没有把-直接用在数字左边这种情况。代价就是，既然-总是一个动词，则需要另一个字符_（下划线）来拼写负数。

_用于拼写数字，放在有0 1 2 3 4 5 6 7 8 9，. 以及表示指数符号的e组成的数字前；表示一个负数。

记住：数字前面的 $\cdot_ \cdot$ 是数字的一部分，表示它是负数，而-总是一个动词，不是数字的一部分。

--（两个下划线）表示负无穷，是一个数字。

第 17 章

原语

原语是由系统定义的一个单词。例如，+是一个原语。原语的含义是固定的，不能改变。

原语可以由一个图形字符（如+）拼写的，或者用图形字符加上一个表示变形的字符（点号或冒号）拼写，如+.或者+:

原语也可以由一个或多个字母后跟一个点号或冒号拼写。例如，i.是一个原语，根据它的使用方式的不同，被称为**生成索引**或**求索引**。